

SNOWFLAKE

STUDY GUIDE

Secure Data Sharing & Storage Billing

- Topic 1: Secure Data Sharing — Summary, Syntax & Q&A;
- Topic 2: Storage Billing — Summary & Q&A;
- Quick Reference Cheat Sheets

TOPIC 1: SECURE DATA SHARING

What is Secure Data Sharing?

Secure Data Sharing allows one Snowflake account to grant **read-only access** to selected database objects to another Snowflake account — **without transferring any data between accounts**. It is enabled by Snowflake's Global Services Layer.

Key Roles

Role	Description
Data Provider	Owns and shares the data. Creates and manages the Share object.
Data Consumer	Receives read-only access. Creates a database from the Share.

How It Works

- Enabled by the **Global Services Layer** — no physical data movement
- Consumer reads directly from the provider's storage
- Consumer pays **compute costs only** — zero storage costs
- Changes and revocations take effect **instantly**

What Can Be Shared?

Object	Shareable?	Notes
Tables	YES	
External Tables	YES	
Secure Views	YES	Recommended over raw tables
Secure Materialized Views	YES	
Secure UDFs	YES	
Regular Views	NO	Must be designated SECURE first
Regular UDFs	NO	Must be designated SECURE first

Snowflake strongly recommends sharing Secure Views over Tables — they hide the view definition and protect against inadvertently exposing source table data.

The Share Object

A **Share** is an account-level object created by the data provider. It has two configurations:

- **Grants** — which database objects to share (WHAT to share)
- **Accounts** — which consumer accounts can access the share (WHO to share with)

Provider Setup — Step by Step

1. Create the Share object (requires ACCOUNTADMIN or CREATE SHARE privilege)
2. Grant USAGE on the **Database** to the share (only ONE database per share)
3. Grant USAGE on one or more **Schemas** to the share
4. Grant SELECT on specific **Database Objects** (tables, views, etc.)
5. Add consumer **account(s)** to the share (virtually unlimited)

Consumer Setup — Step by Step

1. Use SHOW SHARES to list all available shares
2. Use DESCRIBE SHARE to inspect share contents
3. Create a **database from the share** (only ONE database per share)
4. Query shared objects as if they were native tables — including JOINS

Key Provider Behaviors

Behavior	Detail
Objects added to share	Immediately available to ALL consumers
New objects in shared DB	NOT auto-added — must be explicitly granted
Future grants	Cannot be used in shares
DBs per share	ONE database only (hard limit)
Number of shares	No hard limit
Number of consumer accounts	No hard limit
Role required	ACCOUNTADMIN (delegable via CREATE SHARE)
Revoking access	Instant — no grace period

Key Consumer Behaviors

Behavior	Detail
Databases per share	ONE database per share
Time Travel	NOT available on shared tables
Cloning	Cannot clone shared tables
Creating objects	Cannot create objects inside shared database
Resharing	Cannot reshare shared database objects
Role required	ACCOUNTADMIN (delegable via IMPORT SHARE)

Cross-Region Sharing

Secure Data Sharing only works between accounts in the **same region and cloud platform**. To share across regions:

1. Replicate the database into an account in the **same region as the consumer**
2. Create a share from the **replicated database**

Reader Accounts (Managed Accounts)

For consumers who do **not have a Snowflake account**, the provider can provision a Reader Account — a limited account whose sole purpose is to query shared data.

Aspect	Detail
What it is	Limited Snowflake account provisioned by the provider
Purpose	Query and view shared database objects only
Access given	Account URL, username, and password
Limitations	Cannot insert data, cannot copy data in — read only
Share restriction	Can only access shares from the provider that created it
Billing	ALL compute costs billed back to the PROVIDER
Soft limit	20 reader accounts per provider (can be raised via Support)
Agreement needed	NO — consumer does not need a Snowflake agreement

Supported Editions

Edition	Supported?
Standard	YES
Enterprise	YES
Business Critical	YES
Virtual Private Snowflake	NO

TOPIC 1: SYNTAX REFERENCE

Provider — Create & Configure Share

```
-- 1. Create a share
CREATE SHARE my_share;

-- 2a. Grant access to the DATABASE (only one DB per share)
GRANT USAGE ON DATABASE my_database TO SHARE my_share;

-- 2b. Grant access to the SCHEMA
GRANT USAGE ON SCHEMA my_database.my_schema TO SHARE my_share;

-- 2c. Grant access to a TABLE
GRANT SELECT ON TABLE my_database.my_schema.my_table TO SHARE my_share;

-- Grant a SECURE VIEW
GRANT SELECT ON VIEW my_database.my_schema.my_secure_view TO SHARE my_share;

-- Grant a SECURE MATERIALIZED VIEW
GRANT SELECT ON MATERIALIZED VIEW my_database.my_schema.my_smv TO SHARE my_share;

-- Grant a SECURE UDF
GRANT USAGE ON FUNCTION my_database.my_schema.my_udf(VARCHAR) TO SHARE my_share;

-- 3. Add consumer account(s)
ALTER SHARE my_share ADD ACCOUNTS = consumer_org.consumer_account;

-- Add multiple accounts
ALTER SHARE my_share ADD ACCOUNTS = consumer_org.acct_1, consumer_org.acct_2;

-- Remove a consumer account
ALTER SHARE my_share REMOVE ACCOUNTS = consumer_org.consumer_account;
```

Provider — Revoke & Manage

```
-- Revoke a table from the share
REVOKE SELECT ON TABLE my_database.my_schema.my_table FROM SHARE my_share;

-- Revoke schema access
```

```
REVOKE USAGE ON SCHEMA my_database.my_schema FROM SHARE my_share;

-- Show all grants TO a share
SHOW GRANTS TO SHARE my_share;

-- Show all accounts added to a share
SHOW GRANTS OF SHARE my_share;

-- Drop the share entirely
DROP SHARE my_share;
```

Create Secure Objects

```
-- Create a SECURE VIEW (recommended for sharing)

CREATE OR REPLACE SECURE VIEW my_db.my_schema.regional_sales AS

SELECT region, product_category, SUM(sales_amount) AS total_sales

FROM my_db.my_schema.orders

GROUP BY region, product_category;

-- Create a SECURE MATERIALIZED VIEW

CREATE OR REPLACE SECURE MATERIALIZED VIEW my_db.my_schema.sales_mv AS

SELECT product_id, SUM(sales_amount) AS total

FROM my_db.my_schema.sales GROUP BY product_id;

-- Create a SECURE UDF

CREATE OR REPLACE SECURE FUNCTION my_db.my_schema.mask_email(email VARCHAR)

RETURNS VARCHAR LANGUAGE JAVASCRIPT AS

$$ return EMAIL.replace(/^(.{2}).*(@.*)$/, '$1***$2'); $$;
```

Consumer — Discover & Access

```
-- List all available shares

SHOW SHARES;

-- Describe share contents

DESCRIBE SHARE provider_org.provider_account.my_share;

-- Create database from share (only ONE per share)

CREATE DATABASE my_shared_db
```

```
FROM SHARE provider_org.provider_account.my_share;

-- Query shared data

SELECT * FROM my_shared_db.my_schema.my_table;

-- JOIN shared data with own native tables

SELECT s.customer_id, s.customer_name, o.order_id, o.order_amount
FROM my_shared_db.my_schema.customers s
JOIN my_own_db.my_schema.orders o ON s.customer_id = o.customer_id;
```

Reader Accounts & Delegation

```
-- Create a reader account

CREATE MANAGED ACCOUNT reader_acct_1

ADMIN_NAME = 'admin_user'

ADMIN_PASSWORD = 'SecurePass123!'

TYPE = READER

COMMENT = 'Reader account for client XYZ';

-- View all reader accounts

SHOW MANAGED ACCOUNTS;

-- Add reader account to share

ALTER SHARE my_share ADD ACCOUNTS = <reader_account_locator>;

-- Drop a reader account

DROP MANAGED ACCOUNT reader_acct_1;

-- Delegate CREATE SHARE privilege

GRANT CREATE SHARE ON ACCOUNT TO ROLE data_share_manager;

-- Delegate IMPORT SHARE privilege

GRANT IMPORT SHARE ON ACCOUNT TO ROLE data_consumer_role;
```

TOPIC 1: INTERVIEW Q&A;

Q1. What is Secure Data Sharing and how does it work?

Secure Data Sharing allows one Snowflake account (the data provider) to grant read-only access to selected database objects to another account (the data consumer) without physically transferring any data. It works through Snowflake's Global Services Layer, which exposes the provider's storage to the consumer's queries. The consumer reads data directly from the provider's storage — no data movement occurs. The consumer pays compute costs only, not storage costs.

Q2. What is the difference between a Data Provider and a Data Consumer?

A Data Provider owns the data and creates the Share object — they control which objects are shared and which accounts can access them, and can revoke access any time. A Data Consumer receives read-only access, creates a database from the share, and can query shared objects as if native to their own account. They cannot insert, update, clone, or reshare the shared data.

Q3. Why does Snowflake recommend sharing Secure Views over Tables?

When sharing with external organizations, exposing raw tables risks revealing more data than intended. Secure Views:

- Hide the view definition — consumers cannot see the underlying table structure
- Protect against inadvertent data exposure by filtering or masking sensitive columns
- Add an extra security layer appropriate for cross-organization sharing
- Regular views must be designated SECURE before they can be shared at all

Q4. What objects can be included in a Snowflake Share?

Tables, External Tables, Secure Views, Secure Materialized Views, and Secure UDFs. Regular views and UDFs cannot be shared unless explicitly designated as SECURE first.

Q5. A new table is created in a shared database. Is it visible to consumers?

No. New objects are NOT automatically added to a share. The provider must explicitly grant SELECT on the new object to the share. Future grants cannot be used in shares — every object must be individually and explicitly granted.

Q6. What happens when a provider revokes access to a shared object?

Access is instantly removed — there is no grace period. If a provider removes a grant, drops a shared object, or removes a consumer account from a share, the consumer loses access immediately.

Q7. What are the key limitations for data consumers?

Consumers cannot:

- Create only ONE database per share
- Use Time Travel on shared tables
- Clone shared tables
- Create objects inside the shared database

- Reshare shared database objects with other accounts

Q8. What is a Reader Account and when would you use it?

A Reader Account (Managed Account) is a limited Snowflake account provisioned by the provider for organizations that do not have their own Snowflake account. Key facts:

- Consumer gets a URL, username, and password — no Snowflake agreement needed
- Can only query shared objects from the provider that created it
- Cannot insert data or create objects
- ALL compute costs are billed back to the PROVIDER
- Soft limit of 20 per provider (raisable via Snowflake Support)

Q9. How do you share data with a consumer in a different cloud region?

Cross-region sharing is not natively supported. The workaround is: (1) Replicate the database into a Snowflake account in the same region as the consumer, then (2) Create a share from that replicated database. This combines Replication and Secure Data Sharing features.

Q10. Who pays compute costs — provider or consumer?

It depends on the consumer type:

- Standard consumer account — the CONSUMER pays compute costs, zero storage costs
- Reader account — the PROVIDER pays ALL compute costs for the reader account

TOPIC 2: STORAGE BILLING

How Snowflake Calculates Storage Costs

Storage costs are calculated **monthly** based on the **average number of on-disk bytes** stored each day in your Snowflake account.

Every 24 hours

--> Snowflake calculates average data stored that day

End of Month

--> Snowflake averages ALL daily figures across the month

Final Bill

--> Monthly average (TB) x flat rate per TB

Two Areas That Contribute to Storage Costs

1. Database Table Data

ALL three stages of the data lifecycle contribute to billing:

Stage	Description	Billed?
Active Data	Most current, up-to-date micro-partitions	YES
Time Travel	Older micro-partitions retained for recovery (0-90 days)	YES
Fail-Safe	Snowflake-managed 7-day retention after Time Travel	YES

Important: A 90-day retention period gives powerful recovery capabilities but can SIGNIFICANTLY increase storage billing. Always weigh the pros and cons when setting retention periods.

2. Internal Stages

- Any files stored in internal stages contribute to storage costs
- Unlike table data, you have **direct control** over stage files
- Risk: uploading large uncompressed files and forgetting to delete them
- Best practice: clean up stage files immediately after loading

What Affects the Rate Per Terabyte?

Factor	Options
Cloud Provider	AWS, Azure, or GCP

Region	e.g. AWS London (\$42/TB) vs AWS AP Mumbai (higher rate)
Pricing Plan	On-Demand (pay-as-you-go) or Capacity (pre-purchased, cheaper)

Billing Examples (as of Nov 2021)

Example	Region	Plan	Avg Stored	Rate/TB	Monthly Bill
Example 1	AWS London	On-Demand	15 TB	\$42	\$630
Example 2	AWS AP Mumbai	On-Demand	14 TB	\$46	\$644

Key insight: Example 2 stores LESS data (14TB) but costs MORE (\$644) than Example 1 (15TB at \$630) — purely because of a different region. Region selection has a real and significant financial impact.

Cost Management Strategies

Action	Impact
Set high retention period (90 days)	Powerful recovery but HIGHER storage cost
Set retention period to 0	Disables Time Travel, reduces cost
Use Transient tables	No Fail-Safe, max 1-day Time Travel — lower cost
Use Temporary tables	Session-only, no Fail-Safe — lowest cost
Leave large files in internal stages	Unnecessary ongoing storage billing
Clean up stage files after loading	Reduces storage costs immediately
Shorter retention on large busy tables	Best balance of protection vs cost

Table Type Comparison — Storage Cost

Table Type	Time Travel	Fail-Safe	After Deletion	Storage Cost
Permanent	0-90 days	7 days	Gone after 7-day Fail-Safe	Highest
Transient	0-1 day	NONE	Gone after Time Travel	Lower
Temporary	0-1 day	NONE	Gone when session ends	Lowest

Continuous Data Protection — Full Recap

Feature	Duration	Access	Storage Cost
Time Travel	0-90 days (config)	Self-service SQL	Yes — during retention

Fail-Safe	7 days (fixed)	Snowflake Support only	Yes — 7 days
Cloning	N/A	Instant zero-copy	Zero until data diverges
Replication	Ongoing sync	Full access in target account	Yes — full physical copy

TOPIC 2: INTERVIEW Q&A;

Q1. How does Snowflake calculate monthly storage costs?

In two steps: Every 24 hours, Snowflake calculates the average data stored that day. At the end of the month, it averages all daily figures. That monthly average (in TB) is multiplied by a flat rate per terabyte. This approach fairly accounts for data added or removed mid-month.

Q2. What two areas contribute to Snowflake storage costs?

1. Database Table Data — ALL stages of the data lifecycle:

- Active micro-partitions (current data)
- Time Travel micro-partitions (older versions retained for recovery)
- Fail-Safe micro-partitions (7-day Snowflake-managed retention)

2. Internal Stages — any files stored in internal stages contribute to billing. Unlike table data, the user has direct control and should clean up files after loading to avoid unnecessary costs.

Q3. What three factors affect the per-terabyte storage rate?

Cloud Provider (AWS, Azure, GCP), Region (e.g. AWS London vs AWS AP Mumbai), and Pricing Plan (On-Demand or Capacity). Two accounts storing identical data can have very different monthly bills based solely on these factors.

Q4. Why can a 90-day retention period significantly increase storage costs?

Because ALL three stages of the data lifecycle are billed. With 90-day retention, every time data is updated or deleted the old micro-partitions are kept for up to 90 days in Time Travel storage. After that, 7 more days of Fail-Safe are added. For frequently updated tables this means storing many versions simultaneously. Match retention periods to actual recovery needs rather than always maximising them.

Q5. Two accounts store different amounts of data but the second pays more. Why?

Regional pricing differences. For example: AWS London 15TB x \$42 = \$630, while AWS AP Mumbai 14TB x \$46 = \$644. The Mumbai account stores less data but pays more because the per-TB rate is higher in that region. Region selection has a real financial impact and should be factored into account setup decisions.

Q6. How do Transient and Temporary tables help reduce storage costs?

Both types reduce costs by limiting or eliminating data protection stages:

- Transient tables — max 1-day Time Travel, NO Fail-Safe. Data leaves storage faster.
- Temporary tables — session-scoped only. Auto-dropped when session ends, no Fail-Safe.
- Use these for staging tables, ETL intermediate data, or any non-critical data.

Q7. What is the difference between On-Demand and Capacity pricing?

On-Demand: pay as you go, no upfront commitment, higher per-TB rate — good for variable workloads. Capacity: pre-purchased storage commitment, lower per-TB rate — better for predictable large-scale storage needs.

Q8. How would you reduce storage costs for a large frequently-updated table?

Several strategies:

- Reduce the retention period on that specific table (child overrides parent)
- Use Transient tables for non-critical intermediate versions
- Clean up internal stage files immediately after every load cycle
- Monitor storage per table using INFORMATION_SCHEMA.TABLE_STORAGE_METRICS
- Apply long retention only to the most critical tables

QUICK REFERENCE CHEAT SHEETS

Secure Data Sharing — Cheat Sheet

Concept	Key Detail
Data Provider	Creates and manages the Share object
Data Consumer	Creates a database from the Share
Data transfer	NONE — zero-copy via Global Services Layer
Consumer storage cost	NONE — only pays compute
Objects shareable	Tables, ext. tables, secure views, secure MVs, secure UDFs
Views must be	SECURE before they can be shared
DBs per share	ONE database only
DBs per consumer	ONE database created per share
New objects auto-shared	NO — must be explicitly granted
Future grants in shares	NOT supported
Time Travel on shared	NOT available
Clone shared tables	NOT allowed
Resharing	NOT allowed by consumers
Cross-region sharing	Requires replication first
VPS support	NOT supported
Reader account limit	20 soft limit — raisable via Support
Reader account billing	ALL compute billed to PROVIDER
Role to create share	ACCOUNTADMIN (delegable)
Role to import share	ACCOUNTADMIN (delegable)
Revoking access	Instant — no grace period

Storage Billing — Cheat Sheet

Concept	Key Detail
Billing frequency	Monthly
Calculation method	Daily average -> Monthly average x rate per TB
Two billing areas	Database table data + Internal stages

Table data includes	Active + Time Travel + Fail-Safe micro-partitions
Rate factors	Cloud provider + Region + Pricing plan
On-Demand AWS London	\$42/TB (as of Nov 2021)
Retention period impact	Higher retention = more storage cost
Cheapest table type	Temporary (session only, no Fail-Safe)
Stage file risk	Forgotten large files = unnecessary ongoing billing
Best cost strategy	Match retention period to actual recovery needs
After Fail-Safe expires	Data is PERMANENTLY and IRREVERSIBLY deleted

End of Study Guide